UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ESPECIFICAÇÃO LÉXICA DA LINGUAGEM LUMINA

Checkpoint 1 — Especificação Léxica e Analisador Léxico

Linguagem-Alvo do Compilador: TAM (*Triangle Abstract Machine*)

Uberlândia - MG
2026

1. Visão Geral da Linguagem

A linguagem **Lumina** é uma linguagem de programação estaticamente tipada, estruturada e imperativa, projetada para servir como núcleo didático na construção de compiladores com suporte à máquina virtual TAM (*Triangle Abstract Machine*).

O escopo da linguagem contempla:

- **Tipos primitivos:** `int` (inteiro de 32 bits), `double` (ponto flutuante de dupla precisão), `bool` (booleano), `char` (caractere Unicode básico) e `string` (sequência imutável de caracteres).
- **Procedimentos e Funções:** Declaração de funções com tipos de retorno ou `void`, parâmetros tipados, chamadas e instruções de retorno (`return`) com e sem valor.
- **Estruturas de Controle de Fluxo:** Seleção condicional (`if`, `else`) e repetição (`while`).
- **Operações:** Aritmética básica, comparações relacionais, operadores lógicos booleanos e atribuição simples.

2. Resumo das Categorias de Tokens

2.1 Tabela Consolidada das 5 Categorias

Token	Notação (Regex/EBNF)	Exemplos Válidos	Decisão de Projeto
Identificador	`(letra | "_") (letra | dígito | "_")*`	`total`, `x1`, `_contador`	Estritamente *case-sensitive*; `_` permitido; limite de 255 caracteres.
Palavra reservada	Padrão de identificador + busca	`if`, `while`, `int`	Lista fechada de 13 palavras-chave minúsculas.
String	`" (caractere ≠ " | escape)* "`	`"ok"`, `"com \"escape\""`	Escapes aceitos; erro se encontrar `\n` ou EOF antes de fechar.
Operador	1 ou 2 caracteres	`=`, `==`, `<=`, `+`, `&&`	Formas simples e compostas; desambiguação via *Maximal Munch*.
Literal numérico	`dígito+ ("." dígito+)?`	`10`, `3.14`, `0.5`	Suporte a inteiros e decimais; ponto sem dígito posterior é erro.

2.2 Respostas Explícitas às Perguntas Obrigatórias

Questão 1: Alfabeto de Entrada

- **Definição:** O alfabeto de entrada (Σ) é composto pelo conjunto de caracteres imprimíveis da tabela ASCII (códigos decimais 32 a 126), estendido pelos caracteres de controle: espaço em branco (ASCII 32), tabulação horizontal (ASCII 9), quebra de linha Unix (ASCII 10) e retorno de carro (ASCII 13).
- **Codificação:** O código-fonte deve ser codificado em UTF-8. Caracteres fora da faixa ASCII são considerados caracteres inválidos fora de literais de string e geram **erro léxico** imediato, preservando a recuperação do scanner.
- **Caracteres em Strings:** Dentro de literais de cadeia de caracteres (`string`), qualquer caractere ASCII imprimível é permitido, incluindo caracteres especiais escapados (`\"` , `\\` , `\n` , `\t` , `\r`).

Questão 2: Sensibilidade a Caixa (*Case-Sensitivity*)

- A linguagem Lumina é **estritamente sensível a maiúsculas e minúsculas**.
- As palavras reservadas são obrigatoriamente escritas em **letras minúsculas** (ex.: `while` , `if` , `return`).
- Os identificadores preservam a distinção de caixa: `total` , `Total` e `TOTAL` representam três identificadores distintos.
- Uma palavra como `While` ou `IF` não é reconhecida como palavra reservada, mas sim como um identificador comum válido.

Questão 3: Delimitação de Espaços em Branco e Comentários

- **Espaços em Branco:** Espaços, tabulações e quebras de linha atuam como delimitadores léxicos e são descartados pelo scanner. Atualizam contadores de linha/coluna, mas não produzem tokens.
- **Comentários de Linha:** Iniciados por `//` até o primeiro `\n` ou EOF. Conteúdo ignorado.
- **Comentários de Bloco:** Iniciados por `/*` e finalizados por `*/` . Não são aninhados. Caso o EOF seja atingido antes do fechamento, um **erro léxico** é reportado.

Questão 4: Regra de Desambiguação (*Maximal Munch*)

O scanner adota rigidamente a política do casamento mais longo possível (*maximal munch*). Diante de uma sequência ambígua, consome o maior prefixo de entrada válido. Por exemplo, `=` seguido de `=` resulta em `==` , e não em duas atribuições. O identificador `while1` é um identificador único, e não a palavra reservada `while` seguida do literal `1` .

Questão 5: Lista Fechada de Palavras Reservadas

A linguagem Lumina define exatamente **13 palavras reservadas**, todas grafadas em minúsculas:

Categoria	Palavras Reservadas	Significado / Uso
Tipos Primitivos	`int`	Inteiro de 32 bits com sinal
	`double`	Ponto flutuante de precisão dupla (64 bits)
	`bool`	Valor lógico booleano
	`char`	Caractere individual
	`string`	Cadeia de caracteres
	`void`	Ausência de retorno de função / procedimento
Controle de Fluxo	`if`	Instrução de seleção condicional
	`else`	Ramo alternativo da condicional
	`while`	Estrutura de repetição condicional
	`return`	Retorno de função ou procedimento
Declaração	`fun`	Declaração de função ou procedimento
Literais Booleanos	`true`	Literal lógico verdadeiro
	`false`	Literal lógico falso

3. Especificação das Categorias de Tokens

3.1 Identificadores (`IDENTIFIER`)

$$letra \rightarrow [a\text{-}zA\text{-}Z]$$

$$dígito \rightarrow [0\text{-}9]$$

$$Identificador \rightarrow (letra \mid "_") \ (letra \mid dígito \mid "_")^*$$

- O identificador deve começar por uma letra maiúscula/minúscula ou pelo caractere sublinhado (`_`).
- Comprimento máximo suportado: 255 caracteres.
- Se a cadeia coincidir com uma entrada da lista fechada de palavras reservadas, ela é emitida como o respectivo token de palavra-chave.

Exemplos Válidos: `total` , `x1` , `_contador` , `taxa_juros_2` .
Exemplos Inválidos: `1total` (inicia com dígito), `taxa-juros` (hífen não permitido).

3.2 Palavras Reservadas

O padrão regular é um subconjunto exato do padrão de identificadores, interceptado pela consulta na tabela de símbolos reservada (`KeywordTable`). Exemplos: `int` , `double` , `while` , `return` , `true` .

3.3 Literais Numéricos (`INT_LITERAL` , `DOUBLE_LITERAL`)

$$INT_LITERAL \rightarrow [0\text{-}9]+$$

$$DOUBLE_LITERAL \rightarrow [0\text{-}9]+ \text{"."} [0\text{-}9]+$$

- Números inteiros são compostos por um ou mais dígitos decimais.
- Números decimais exigem obrigatoriamente um ou mais dígitos antes do ponto, o caractere de ponto `.` e um ou mais dígitos após o ponto.
- Formas com ponto sem dígitos posteriores (ex.: `42.`) constituem **erro léxico** ("Literal decimal malformado: esperava-se dígitos após o ponto").

Exemplos Válidos: `0` , `42` , `3.1415` , `0.5` .
Exemplos Inválidos: `.5` (não inicia com dígito), `42.` (ponto isolado).

3.4 Literais de Cadeia de Caracteres (`STRING_LITERAL`)

$$CaractereNormal \rightarrow [\text{^ " \ \r \n }]$$

$$Escape \rightarrow \text{\ (" | \ | n | t | r)}$$

$$STRING_LITERAL \rightarrow \text{" (CaractereNormal | Escape)* "}$$

- As strings são delimitadas exclusivamente por aspas duplas `"` e suportam sequências de escape convencionais.
- A linguagem Lumina não admite strings multilinha literais sem escape. O encontro de uma quebra de linha física (`\n`) ou `EOF` antes do fechamento das aspas duplas constitui um **erro léxico**.

Exemplos Válidos: `"ok"` , `"Mensagem: \"Sucesso!\"\n"` .
Exemplos Inválidos: `"string sem fim` (quebra de linha prematura).

3.5 Literais de Caractere (`CHAR_LITERAL`)

$$CHAR_LITERAL \rightarrow \text{' (CaractereNormalChar | EscapeChar) '}$$

Delimitado por aspas simples `'` . Contém exatamente um caractere ou uma sequência de escape válida.
Exemplos Válidos: `'a'` , `'9'` , `'\n'` , `'\''` .

3.6 Operadores

A linguagem define operadores de 1 e 2 caracteres, classificados por categoria funcional:

Operador	Tipo de Token	Categoria	Descrição
`+`	`PLUS`	Aritmético	Adição
`-`	`MINUS`	Aritmético	Subtração
`*`	`STAR`	Aritmético	Multiplicação
`/`	`SLASH`	Aritmético	Divisão
`%`	`PERCENT`	Aritmético	Resto da divisão inteira
`=`	`EQUAL`	Atribuição	Atribuição simples de valor
`==`	`EQUAL_EQUAL`	Relacional	Comparação de igualdade
`!=`	`BANG_EQUAL`	Relacional	Comparação de diferença
`<`	`LESS`	Relacional	Menor que
`<=`	`LESS_EQUAL`	Relacional	Menor ou igual que
`>`	`GREATER`	Relacional	Maior que
`>=`	`GREATER_EQUAL`	Relacional	Maior ou igual que
`&&`	`AMPERSAND_AMPERSAND`	Lógico	Conjunção lógica (E)
`||`	`PIPE_PIPE`	Lógico	Disjunção lógica (OU)
`!`	`BANG`	Lógico	Negação lógica (NÃO)

3.7 Delimitadores e Pontuação

Símbolo	Tipo de Token	Significado
`(`	`LEFT_PAREN`	Abertura de parênteses
`)`	`RIGHT_PAREN`	Fechamento de parênteses
`{`	`LEFT_BRACE`	Abertura de bloco de comandos
`}`	`RIGHT_BRACE`	Fechamento de bloco de comandos
`[`	`LEFT_BRACKET`	Abertura de colchetes
`]`	`RIGHT_BRACKET`	Fechamento de colchetes
`;`	`SEMICOLON`	Terminador de instrução
`,`	`COMMA`	Separador de argumentos ou itens
`:`	`COLON`	Anotador de tipo (se aplicável)

3.8 Tokens Especiais de Controle

- `EOF` : Emitido ao alcançar o fim do fluxo de entrada de caracteres.
- `ERROR` : Emitido para indicar um caractere inválido ou malformação léxica, preservando o lexema defeituoso.

4. Política de Tratamento e Recuperação de Erros

Em concordância com os princípios de robustez:

1. **Não Interrupção Abrupta:** O scanner nunca lança exceções não tratadas durante o processo de tokenização normal. Erros léxicos não devem derrubar o compilador.
2. **Coleta e Localização Exata:** Todo erro léxico é encapsulado em um objeto `LexicalError`, contendo linha, coluna, mensagem descritiva e o lexema falho.
3. **Recuperação em Modo Pânico Local:**
    - **Caractere Inválido:** Consome o caractere defeituoso, emite `ERROR` e reinicia a leitura.
    - **String Não Fechada:** Encerra o token como `ERROR` ao encontrar `\n` ou `EOF`.
    - **Comentário Não Fechado:** Ao atingir `EOF` dentro de `/*`, registra o erro léxico.

5. Diagramas dos Autômatos Finitos Determinísticos

As representações a seguir descrevem as funções de transição de estado projetadas para o analisador léxico.

5.1 Identificador e Palavra Reservada

[Início] → `[a-zA-Z_]` → **S1**

S1 → `[a-zA-Z0-9_]` → **S1** *(Laço Maximal Munch)*

S1 → `outro` → **[Busca]**

[Busca] → `na Tabela?` → **KEYWORD** *(Se sim)*

[Busca] → `na Tabela?` → **IDENTIFIER** *(Se não)*

5.2 Literais Numéricos (Inteiro e Ponto Flutuante)

[Início] → `dígito [0-9]` → **S1**

S1 → `dígito [0-9]` → **S1**

S1 → `'.' + lookahead(díg)` → **S2**

S1 → `'.' + lookahead(!díg)` → **ERRO** *(Ponto sem fração)*

S2 → `dígito [0-9]` → **S3**

S3 → `dígito [0-9]` → **S3**

S1 → `outro / delim.` → **INT_LIT**

S3 → `outro / delim.` → **DOUBLE_LIT**

5.3 Literais String e Sequências de Escape

[Início]	→	`'"'`	→	**S1**	
S1	→	`caractere normal`	→	**S1**	
S1	→	`'\' (escape)`	→	**S_ESC**	
S_ESC	→	`válido (" \ n t r)`	→	**S1**	
S1	→	`'"'`	→	**STRING_LIT**	
S1	→	`'\n' / EOF`	→	**ERRO**	*(Quebra proibida)*

5.4 Operadores Relacionais e Lógicos (Maximal Munch)

[Início]	→	`'='`	→	**S_EQ**	
S_EQ	→	`'='`	→	**EQUAL_EQ**	*(==)*
S_EQ	→	`outro`	→	**EQUAL**	*(=)*
[Início]	→	`'<'`	→	**S_LT**	
S_LT	→	`'='`	→	**LESS_EQ**	*(<=)*
S_LT	→	`outro`	→	**LESS**	*(<)*
[Início]	→	`'&'`	→	**S_AND**	
S_AND	→	`'&'`	→	**AND_AND**	*(&&)*
S_AND	→	`outro`	→	**ERRO**	

5.5 Descarte de Espaços e Comentários

[Início]	→	`espaço / \n`	→	**[Início]**	*(Descarta)*
[Início]	→	`'/'`	→	**S_SLASH**	
S_SLASH	→	`'/'`	→	**COM_LIN**	*(Linha // até \n)*
S_SLASH	→	`'*'`	→	**COM_BLOC**	*(Bloco /*)*
COM_BLOC	→	`'*' seguido de '/'`	→	**[Início]**	*(Fecha bloco)*

6. Exemplo Sintético de Código Válido em Lumina

Abaixo consta um exemplo canônico demonstrando as estruturas da linguagem, incluindo declaração de funções, uso de tipos primitivos, estruturas de repetição, condicionais e literais.

```
// Exemplo canônico de rotina em Lumina
fun int calcularFatorial(int n) {
    /* Trata o caso base de recursão ou iteração
       Calcula o fatorial acumulado */
    int resultado = 1;
    int contador = n;

    while (contador > 1) {
        resultado = resultado * contador;
        contador = contador - 1;
    }

    if (resultado >= 1 && resultado != 0) {
        // Retorna o resultado apurado
        return resultado;
    } else {
        return 0;
    }
}

fun void main() {
    int valor = 5;
    double taxa = 3.1415;
    string mensagem = "Fatorial calculado: \"OK\"\n";
    int f = calcularFatorial(valor);
}
```